

Generative AI Elevate your Software Development Career

Maulana Hafidz Ismail

25 Oktober 2025

Contents

1	Generative AI and Software Development	1
1.1	Beberapa Tokoh	1
1.2	Definisi Generative AI	1
1.3		1
1.4	Tren Masa Depan untuk AI dan Pengembangan Perangkat Lunak	2
1.5	Fase dalam SDLC	2
1.6	Penggunaan AI dalam SDLC	3
1.7	Penerapan AI di Setiap Fase	3
1.8	Definisi dan Konsep Utama LLM	3
1.9	Peran LLM dalam Pengembangan Perangkat Lunak	3
1.10	Kemampuan LLM	4
1.11	Pertimbangan Etis dan Tantangan	4
1.12	Tren Masa Depan untuk LLM	4
1.13	Pengertian Natural Language Processing (NLP)	4
1.14	Komponen dan Teknik NLP	4
1.15	Perpustakaan dan Alat NLP	5
1.16	Penggunaan NLP dalam Pengembangan Perangkat Lunak	5
1.17	Implikasi Etis dari NLP	6
1.18	Contoh Gen AI untuk Software Dev	6
1.19	Apa itu Token dalam API AI?	6
1.19.1	Token Tools	7
1.19.2	Bagaimana cara menghitung Token?	7
1.19.3	Petunjuk OpenAI dalam Menghitung Token	8
1.19.4	Contoh Perhitungan Token	8
1.20	Penggunaan Gen AI dalam Pengembangan Perangkat Lunak	9
1.21	Automasi dalam Testing	10
1.22	Contoh Alat Gen AI	10
1.23	memanfaatkan AI untuk Bantuan Teknis	10
1.24	Implementasi Design Patterns dengan Pendekatan AI	10
1.25	AI dan Arsitektur Perangkat Lunak	10
1.26	Tren Masa Depan dalam Pengembangan Perangkat Lunak	11
1.27	Pemanfaatan AI dalam Proses Desain Perangkat Lunak	11
1.28	Interaksi dengan Pengembang	11
1.29	Contoh Penggunaan AI	11
1.30	Alat dan Fitur AI	12
1.31	Penggunaan AI dalam Pengembangan Perangkat Lunak	12
1.32	Alat AI Populer untuk Pengembangan Perangkat Lunak	12
2	Generative AI for Software Development Workflows and its Considerations	13
3	Final Project and Final Exam	14

1 Generative AI and Software Development

1.1 Beberapa Tokoh

- Paul Pardi - Ahli materi di SkillUp Technology.
- Shubhra Das - Ahli IT di SkillUp Tech.
- Upkar Lidder - Manajer produk.
- Michael Rudnitski - Insinyur perangkat lunak di IBM.
- Ramesh Sannareddy - Direktur di Mongo Factory.

1.2 Definisi Generative AI

Generative AI adalah penggunaan teknik artificial intelligence untuk menghasilkan kode dan konten baru, seperti gambar, teks, atau musik yang menyerupai data yang ada.

1.3

- Automasi
Generative AI dapat digunakan untuk mengotomatiskan tugas-tugas berulang, mengurangi usaha manual, dan meningkatkan produktivitas. Contoh: Menggunakan AI untuk mengotomatiskan pengujian perangkat lunak.
- Code Optimization
AI dapat menganalisis basis kode yang besar untuk mengidentifikasi area yang dapat dioptimalkan guna meningkatkan kinerja atau mengurangi penggunaan memori. Contoh: Menggunakan algoritma AI untuk menyarankan perbaikan pada kode.
- Bug Detection
Dengan machine learning, model dapat dilatih untuk mengidentifikasi bug dan kerentanan umum dalam kode. Contoh: Menggunakan model untuk menganalisis pola dan laporan bug sebelumnya.
- Enhancing User Experience
Natural Language Processing (NLP) dapat digunakan untuk menganalisis umpan balik pengguna dan mengekstrak wawasan berharga. Contoh: Menggunakan NLP untuk memahami sentimen pengguna terhadap aplikasi.

- **Augmenting Creativity**
Generative models, seperti Generative Adversarial Networks (GANs), dapat digunakan untuk menciptakan desain baru dan inovatif. Contoh: Menggunakan GANs untuk menghasilkan antarmuka pengguna yang menarik.
- **Smart Documentation**
Sistem dokumentasi pintar dapat menganalisis kode dan komentar untuk menghasilkan dokumentasi yang deskriptif secara otomatis. Contoh: Menggunakan AI untuk membuat dokumentasi dari kode yang ada.

1.4 Tren Masa Depan untuk AI dan Pengembangan Perangkat Lunak

- **Efficiency Enhancement:** Generative AI dapat meningkatkan efisiensi dengan mengotomatiskan tugas yang memakan waktu.
- **Creative Collaboration:** Kolaborasi antara manusia dan model generatif dapat menghasilkan ide-ide baru.
- **Low-Code/No-Code Platforms:** Platform ini memungkinkan pengguna membangun aplikasi dengan sedikit pengetahuan coding.
- **Explainable AI:** AI yang dapat menjelaskan proses pengambilan keputusan model.
- **Intelligent Assistants:** Asisten cerdas yang lebih maju akan membantu pengembang selama siklus hidup pengembangan perangkat lunak.
- **Ethical AI Development:** Penekanan pada praktik pengembangan AI yang etis dan adil.

1.5 Fase dalam SDLC

- **Requirements Gathering and Analysis:** AI membantu mengotomatiskan analisis kebutuhan pengguna dan mengidentifikasi pola dalam dataset besar.
- **Design:** AI memberikan rekomendasi untuk arsitektur perangkat lunak yang optimal dan menghasilkan dokumentasi desain berdasarkan data historis.
- **Development:** AI mengotomatiskan tugas pengkodean yang berulang dan membantu dalam tinjauan kode untuk mengidentifikasi bug atau kerentanan.
- **Testing:** AI mengotomatiskan pembuatan dan eksekusi test case, serta menganalisis perubahan kode untuk mengidentifikasi area yang terpengaruh.

- **Deployment:** AI membantu mengotomatiskan proses deployment dan memantau kinerja perangkat lunak yang telah dideploy.
- **Maintenance:** AI menganalisis log dan umpan balik pengguna untuk mengidentifikasi pola dan memprediksi masalah potensial.

1.6 Penggunaan AI dalam SDLC

- **Natural Language Processing (NLP):** Teknologi yang memungkinkan komputer untuk memahami dan memproses bahasa manusia. Dalam konteks ini, NLP digunakan untuk mengekstrak informasi dari kebutuhan pengguna.
- **Machine Learning (ML):** Cabang dari AI yang memungkinkan sistem untuk belajar dari data dan meningkatkan kinerjanya seiring waktu. Dalam SDLC, ML digunakan untuk menganalisis data dan membuat prediksi.

1.7 Penerapan AI di Setiap Fase

- **Requirements Gathering:** NLP digunakan untuk mengekstrak informasi penting dari kebutuhan pengguna dan mengidentifikasi pola.
- **Design and Architecture:** AI menganalisis data untuk memberikan rekomendasi desain yang lebih baik dan mengidentifikasi pola desain yang sukses.
- **Code Generation:** AI dapat menghasilkan potongan kode berdasarkan basis kode yang ada, mempercepat proses pengkodean.
- **Testing and Quality Assurance:** AI mengotomatiskan pembuatan test case dan membantu dalam prediksi dan pencegahan bug.
- **DevOps:** Integrasi AI dalam DevOps memungkinkan pengambilan keputusan otomatis berdasarkan data real-time, meningkatkan efisiensi dan keandalan.

1.8 Definisi dan Konsep Utama LLM

Large Language Models (LLMs) adalah model AI yang dilatih untuk memahami dan menghasilkan teks yang koheren dan relevan secara kontekstual. Mereka menggunakan teknik deep learning pada dataset besar yang berisi teks dari berbagai sumber.

1.9 Peran LLM dalam Pengembangan Perangkat Lunak

- **Code Generation and Auto-completion:** LLMs membantu dalam menghasilkan kode dan menyarankan penyelesaian kode, mengurangi usaha manual dan mempercepat proses pengkodean.

- Automated Bug Detection and Fixing: LLMs dapat mengidentifikasi bug dalam kode dengan menganalisis sintaks dan semantik, serta memberikan saran perbaikan.

1.10 Kemampuan LLM

- Generating Code Templates and Snippets: LLMs dapat membuat template dan potongan kode berdasarkan kebutuhan spesifik, membantu pengembang menghemat waktu.
- Assisting with Documentation and Comments: LLMs menganalisis fungsi kode untuk menghasilkan dokumentasi dan komentar, meningkatkan keterbacaan kode.

1.11 Pertimbangan Etis dan Tantangan

- Bias Detection: LLMs dapat mewarisi bias dari data pelatihan mereka, sehingga penting untuk mendeteksi dan mengurangi bias ini untuk mencegah hasil yang tidak adil.
- Security and Privacy: Penggunaan LLM harus mempertimbangkan keamanan dan privasi kode untuk melindungi dari akses yang tidak sah.

1.12 Tren Masa Depan untuk LLM

- Integration with IDEs: Penelitian sedang dilakukan untuk mengintegrasikan LLM dengan Integrated Development Environments (IDEs) untuk memberikan akses mudah ke fitur yang didukung LLM.
- Collaboration with Developers: Kerjasama antara pengembang dan LLM akan terus berkembang, dengan pengembang membimbing keluaran LLM selama berbagai tahap pengembangan perangkat lunak.

1.13 Pengertian Natural Language Processing (NLP)

Natural Language Processing adalah teknik komputasi yang digunakan untuk menganalisis, memahami, dan menghasilkan bahasa manusia. Ini menggabungkan linguistik, ilmu komputer, dan kecerdasan buatan untuk memproses data bahasa alami.

1.14 Komponen dan Teknik NLP

- Linguistics Concepts: Dasar dari NLP yang mencakup:
 - Morphology: Studi tentang struktur kata.
 - Syntax: Studi tentang struktur kalimat.
 - Semantics: Studi tentang makna.

- Text Processing and Cleaning Techniques: Langkah penting dalam NLP yang meliputi:
 - Tokenization: Proses membagi teks menjadi unit-unit yang lebih kecil yang disebut tokens (kata, kalimat, atau karakter).
 - Stemming: Teknik yang mengurangi kata ke bentuk dasarnya untuk menyederhanakan kosakata.

1.15 Perpustakaan dan Alat NLP

Beberapa perpustakaan dan alat NLP yang umum digunakan:

- IBM Watson
- Aylien
- Google Cloud NLP API
- NLTK: Perpustakaan Python yang paling populer untuk NLP.
- MonkeyLearn
- Amazon Comprehend
- Stanford Core NLP
- TextBlob
- SpaCy
- GenSim

1.16 Penggunaan NLP dalam Pengembangan Perangkat Lunak

- Sentiment Analysis: Proses menentukan emosi yang diekspresikan dalam teks dengan memberikan skor polaritas pada kata atau frasa.
- Named Entity Recognition (NER): Mengidentifikasi dan mengklasifikasi entitas bernama dalam teks ke dalam kategori yang telah ditentukan.
- Text Classification: Tugas mengkategorikan teks ke dalam label atau kategori yang telah ditentukan.
- Machine Translation: Teknik yang memungkinkan komputer untuk memahami dan menghasilkan bahasa manusia.
- Chatbots: Program komputer yang mensimulasikan percakapan manusia dengan menggunakan teknik NLP untuk memahami pertanyaan pengguna dan menghasilkan respons yang sesuai.

1.17 Implikasi Etis dari NLP

NLP juga menimbulkan kekhawatiran etis terkait privasi, bias, keadilan, transparansi, dan akuntabilitas. Penting untuk memastikan keadilan dalam algoritma untuk mencegah diskriminasi dan menggunakan NLP secara bertanggung jawab dengan mempertimbangkan pertimbangan etis sepanjang siklus pengembangan.

1.18 Contoh Gen AI untuk Software Dev

- ChatGPT adalah karya OpenAI yang dikenal karena kemampuannya menghasilkan teks yang mirip manusia. ChatGPT unggul dalam percakapan alami, penjelasan pertanyaan, dan dukungan penulisan kreatif.
- Watsonx.ai membantu memproses volume besar data teks, menghemat waktu dan sumber daya, serta kemampuannya dalam penerjemahan bahasa memfasilitasi komunikasi dan kolaborasi global.
- GPT-4 adalah model bahasa AI canggih yang membantu menghasilkan teks berkualitas tinggi, meningkatkan penciptaan konten, dan memperkuat tugas pemrosesan bahasa alami.
- Bard adalah chatbot dan alat penciptaan konten terdepan yang dikembangkan oleh Google. Ia memanfaatkan LaMDA, model berbasis transformer, untuk membantu pengembangan perangkat lunak, pertanyaan pemrograman, penulisan konten, penelitian, dan belajar.

1.19 Apa itu Token dalam API AI?

Token adalah bagian penting dari API ChatGPT. Token adalah potongan atau segmen kata. Sebelum memproses prompt, API memecah masukan menjadi token-token individu ini.

Token-token ini tidak selalu sejalan dengan awal atau akhir kata — mereka dapat mencakup spasi di akhir dan bahkan sub-kata.

Token Limits: Biasanya, requests dapat menggunakan hingga 4097 token yang dibagi antara prompt dan completion, tergantung pada modelnya. Misalnya, jika prompt Anda terdiri dari 4000 token, completion Anda dapat berisi maksimal 97 token. Batasan ini merupakan kendala teknis, tetapi seringkali ada cara kreatif untuk mengatasinya, seperti mengompres prompt Anda atau membagi teks menjadi potongan-potongan yang lebih kecil.

Token Pricing: API menyediakan berbagai jenis model dengan harga yang berbeda-beda. Setiap model memiliki rentang kemampuan yang berbeda, dengan "gpt-3.5-turbo" sebagai yang paling canggih. Permintaan yang ditujukan ke model-model ini memiliki harga yang berbeda. Informasi detail mengenai harga token tersedia di halaman API produk.

Exploring Tokens: API memproses kata berdasarkan konteksnya dalam data korpus. GPT-3 mengambil prompt, mengubah input menjadi daftar token, memproses prompt, dan kemudian mengubah token yang diprediksi kembali

menjadi kata-kata sebagai respons. Perlu dicatat bahwa dua kata yang tampak identik dapat dihasilkan sebagai token yang berbeda tergantung pada strukturnya dalam teks. Misalnya, pertimbangkan bagaimana API menghasilkan nilai token untuk kata "red" berdasarkan konteksnya.

1.19.1 Token Tools

Alat yang paling populer di antara semua alat adalah alat tokenizer interaktif OpenAI. Alat ini membantu menghitung jumlah token dan mengamati bagaimana teks dibagi menjadi token.

Jika perlu melakukan tokenisasi teks secara programatik, gunakan Tiktoken, sebuah tokenizer BPE yang cepat dan dirancang khusus untuk model OpenAI.

Perpustakaan lain termasuk paket transformers untuk Python atau paket gpt-3-encoder untuk Node.js.

1.19.2 Bagaimana cara menghitung Token?

Untuk menghitung token saat memanggil OpenAI, ikuti langkah-langkah berikut:

- Identifikasi endpoint API: Tentukan endpoint OpenAI mana yang akan Anda panggil.
- Setiap endpoint mewakili fungsi atau sumber daya spesifik yang disediakan oleh API.
- Periksa dokumentasi API: Akses dokumentasi API yang disediakan oleh penyedia API. Dokumentasi tersebut akan menjelaskan struktur permintaan dan respons API, termasuk header, parameter, atau metode autentikasi yang diperlukan.
- Periksa apakah API memerlukan otentikasi berbasis token: Otentikasi berbasis token melibatkan penambahan token akses ke header permintaan untuk mengotorisasi panggilan API. Token ini biasanya diperoleh dengan mendaftarkan aplikasi ke penyedia API dan mengikuti proses otentikasi mereka.
- Peroleh token akses: Jika otentikasi berbasis token diperlukan, peroleh token akses dengan mengikuti proses otentikasi yang ditentukan oleh penyedia API. Proses ini mungkin melibatkan pendaftaran aplikasi, penyediaan kredensial, dan penerimaan token.
- Hitung token untuk setiap API: Setelah Anda memiliki token akses, hitunglah sebagai satu token untuk setiap panggilan API yang Anda lakukan. Setiap permintaan ke titik akhir API, termasuk token akses dalam header, dihitung sebagai satu token.

- Pantau penggunaan token: Pantau jumlah token yang telah Anda gunakan untuk memastikan Anda tetap dalam batas penggunaan atau kuota yang ditetapkan oleh penyedia API. Beberapa API mungkin membatasi jumlah token yang dapat Anda gunakan dalam periode tertentu.

1.19.3 Petunjuk OpenAI dalam Menghitung Token

Berdasarkan dokumen resmi OpenAI, berikut adalah perkiraan jumlah token:

- 1 token = 4 karakter dalam bahasa Inggris
- 1 token = $\frac{3}{4}$ kata
- 100 token = 75 kata
- 1-2 kalimat = 30 token
- 1 paragraf = 100 token
- 1.500 kata = 2.048 token

1.19.4 Contoh Perhitungan Token

Misalnya, Anda memiliki endpoint OpenAI yang memerlukan permintaan GET untuk mengambil informasi pengguna. Endpoint tersebut memiliki detail sebagai berikut:

- Metode Permintaan: GET
- URL Endpoint: `https://api.example.com/users/userId`
- Parameter Jalan: `userId` (nilai: "123")
- Parameter Query: `includeAddress` (nilai: true)

Menghitung token untuk permintaan ini:

- Metode permintaan 'GET' biasanya memerlukan 1 token.
- URL endpoint tidak mengonsumsi token tambahan.
- Parameter jalur 'userId' dengan nilai '123' tidak mengonsumsi token tambahan.
- Parameter kueri 'includeAddress' dengan nilai 'true' mengonsumsi 2 token (1 token untuk nama parameter dan 1 token untuk nilai parameter).
- Jumlah total token dari langkah 1 hingga 4, jumlah token total untuk permintaan API ini adalah 3 token.

Catatan: Metode penghitungan token yang tepat dapat bervariasi tergantung pada implementasi API spesifik atau penyedia API. Konsultasikan dokumentasi API untuk informasi akurat tentang penggunaan token dan biaya atau batasan yang terkait.

Contoh: Menghitung biaya untuk prompt masukan dan keluaran menggunakan rumus perhitungan biaya sampel.

Prompt masukan: Prompt: “Bisakah Anda memberikan gambaran singkat tentang manfaat berolahraga?”

- Langkah 1:

Tentukan jumlah kata dalam prompt.

Dalam hal ini, prompt memiliki 9 kata.

- Langkah 2:

Hitung biaya untuk prompt berdasarkan jumlah token. Asumsikan biaya per 1000 token adalah \$0,03.

Karena prompt memiliki 9 kata dan tingkat konversi rata-rata adalah 750 kata per 1000 token, biaya prompt = $(9 / 750) * (0,03) = \$0,00036$

- Langkah 3:

Jika model AI menghasilkan output sekitar 500 kata, gunakan rumus yang sama untuk menghitung biaya menghasilkan output. Biaya output = $(500 / 750) * (0,06) = \$0,04$

- Langkah 4:

Hitung biaya total. Biaya total diperkirakan = Biaya prompt + Biaya output = $\$0,00036 + \$0,04 = \$0,04036$

Oleh karena itu, biaya diperkirakan untuk menghasilkan output sekitar 500 kata menggunakan prompt yang diberikan adalah \$0,04036.

Demikian pula, jika aplikasi memanggil API 1000 kali sehari, biayanya ditampilkan sebagai berikut:

Biaya per hari = \$40,36 Biaya per bulan = \$1.210,80

Catatan: Biaya aktual dapat bervariasi tergantung pada jumlah token yang digunakan dalam menghasilkan konten dan jenis model yang digunakan.

1.20 Penggunaan Gen AI dalam Pengembangan Perangkat Lunak

- Code Generation: Generative AI dapat secara otomatis menulis potongan kode berdasarkan kebutuhan input, mengurangi waktu yang dihabiskan pengembang untuk tugas pengkodean yang berulang.
- Bug Detection: AI dapat memindai kode untuk mendeteksi masalah potensial dan menawarkan solusi, membantu pengembang menemukan dan memperbaiki bug lebih cepat.

1.21 Automasi dalam Testing

- Test Case Generation: AI dapat mengotomatiskan pembuatan test case, yang membantu dalam mengidentifikasi dan memperbaiki masalah dengan lebih cepat.
- Automated Testing Tools: Alat seperti Testim dapat menghasilkan dan mengeksekusi test case berdasarkan alur pengguna.

1.22 Contoh Alat Gen AI

- GitHub Copilot: Alat ini membantu pengembang, terutama yang junior, untuk menulis metode, kelas, dan fungsi lebih cepat dengan memberikan saran potongan kode berdasarkan niat fungsi.
- ChatGPT: Dapat digunakan untuk berbagai tugas generative AI, termasuk menghasilkan dokumentasi dari kode.

1.23 memanfaatkan AI untuk Bantuan Teknis

- AI-Powered Code Completion Tools: Alat ini membantu dalam menyelesaikan kode dengan memberikan saran yang akurat, menghemat waktu dan meningkatkan akurasi pengkodean.
- Automated Code Review and Bug Detection: AI dapat menilai kualitas kode, menemukan bug potensial, dan merekomendasikan perbaikan.

1.24 Implementasi Design Patterns dengan Pendekatan AI

- Singleton Design Pattern: Digunakan untuk memastikan hanya ada satu instance dari kelas tertentu, seperti koneksi database.

```
class DatabaseConnection { private static instance;  
    public static getInstance() { ... } }
```

- Observer Pattern: Memungkinkan objek untuk memberi tahu objek lain tentang perubahan status, seperti dalam aplikasi chat. Definisi: Pola desain yang memungkinkan satu objek mengawasi dan merespons perubahan pada objek lain.

1.25 AI dan Arsitektur Perangkat Lunak

- Generating High-Level Architecture from Code: AI dapat menghasilkan arsitektur tingkat tinggi dari kode yang ada, memberikan wawasan tentang struktur sistem. Definisi: Proses menciptakan gambaran umum dari arsitektur perangkat lunak berdasarkan kode yang ada.

- Real-Time Architecture Updates: Memungkinkan pembaruan arsitektur secara langsung untuk meningkatkan kolaborasi dalam proyek pengembangan perangkat lunak. Definisi: Pembaruan yang dilakukan secara langsung untuk mencerminkan perubahan dalam arsitektur.

1.26 Tren Masa Depan dalam Pengembangan Perangkat Lunak

- AI-Driven Tools and Platforms: Alat yang didorong oleh AI akan mendukung pengembang dengan mengotomatiskan tugas dan menghasilkan prototipe.
- Quantum Computing Integration: Integrasi komputasi kuantum dapat merevolusi pengembangan perangkat lunak dengan menyelesaikan masalah kompleks. Penggunaan komputer kuantum untuk memecahkan masalah yang tidak dapat diselesaikan oleh komputer klasik.

1.27 Pemanfaatan AI dalam Proses Desain Perangkat Lunak

- Automasi Desain: AI digunakan untuk menganalisis kebutuhan dan menghasilkan pola desain yang sesuai dengan praktik terbaik, sehingga membuat fase awal desain lebih efisien.
- Pengurangan Kesalahan Manual: Dengan menggunakan AI, tim dapat mengurangi kesalahan manual, meningkatkan konsistensi desain, dan mempercepat proses pengembangan.

1.28 Interaksi dengan Pengembang

- Dialog dengan Developer: Gen AI dapat berkomunikasi dengan pengembang, memungkinkan mereka untuk memberikan deskripsi tentang apa yang ingin dicapai dalam solusi perangkat lunak.
- Pembuatan Alur Logika: AI dapat membuat deskripsi alur logika yang menunjukkan bagaimana data akan bergerak melalui aplikasi dan operasi yang perlu dibuat oleh pengembang.

1.29 Contoh Penggunaan AI

- Analisis Proyek Serupa: Alat seperti OpenAI Codex dapat menghasilkan kode contoh untuk arsitektur tipikal, seperti MVP atau MVVP.
- Prototipe Cepat: Gen AI digunakan untuk membuat desain UI/UX dan prototipe cepat yang dapat ditunjukkan kepada klien.

1.30 Alat dan Fitur AI

- Refact.ai: Alat ini membantu menyusun kode dengan mendeteksi dan merombak blok kode yang berulang, meningkatkan keterbacaan dan kinerja.
- DiagramGPT dan LeonardoAI: Alat ini dapat mengubah deskripsi bahasa alami menjadi diagram arsitektur yang dapat dibangun dan ditingkatkan oleh pengembang.
- Lucidchart: Alat untuk membuat diagram yang dapat mengintegrasikan fitur AI untuk membantu menghasilkan diagram sistem atau jaringan secara otomatis berdasarkan data yang dimasukkan.
- Microsoft Visio: Alat diagram yang juga dapat memanfaatkan fitur AI untuk membantu dalam pembuatan diagram arsitektur dan visualisasi sistem.
- PlantUML: Alat yang memungkinkan pengembang untuk membuat diagram dari kode, yang dapat diintegrasikan dengan pipeline CI/CD untuk memperbarui diagram secara otomatis saat kode berubah.

1.31 Penggunaan AI dalam Pengembangan Perangkat Lunak

- Code Generation: AI dapat secara otomatis menghasilkan potongan kode berdasarkan kebutuhan yang diberikan, mengurangi waktu yang dihabiskan pengembang untuk tugas pengkodean yang berulang.
- Bug Detection: AI dapat memindai kode untuk mendeteksi potensi masalah dan menawarkan solusi, membantu pengembang menemukan dan memperbaiki bug lebih cepat.
- Technical Support: AI meningkatkan efisiensi dukungan teknis dengan menggunakan Natural Language Processing (NLP) untuk memahami dan merespons pertanyaan pengembang.

1.32 Alat AI Populer untuk Pengembangan Perangkat Lunak

- ChatGPT: Alat AI yang digunakan untuk menjawab pertanyaan dan membantu dalam penulisan serta debugging kode.
- GitHub Copilot: Alat AI yang berfungsi sebagai rekan pemrograman, memberikan saran kode secara real-time.
- Tabnine: Alat penyelesaian kode AI yang memberikan saran berdasarkan pola pengkodean yang dipelajari.

2 Generative AI for Software Development Workflows and its Considerations

3 Final Project and Final Exam